

UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO

Elton Felipe Mariano da Fonseca

Sistema de Monitoramento e Análise de Uso de Impressoras

Vídeo de apresentação do Projeto Integrador

<https://www.youtube.com/watch?v=nfwrk72nWPE>

Repositório GitHub

<https://github.com/jacarei-sme/sme-printer>

Jambeiro - SP
2026

UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO

Sistema de Monitoramento e Análise de Uso de Impressoras

Relatório Técnico-Científico apresentado na disciplina de Projeto Integrador para o curso de Ciências de Dados da Universidade Virtual do Estado de São Paulo (UNIVESP).

Jambeiro - SP
2026

FONSECA, Elton Felipe Mariano da. **Sistema de Monitoramento e Análise de Uso de Impressoras**. 00f. Relatório Técnico-Científico. Ciências de Dados – **Universidade Virtual do Estado de São Paulo**. Tutor: Márcio de Oliveira Santiago Filho. Polo Jambeiro, 2026.

RESUMO

A gestão eficiente de recursos é um desafio constante na administração pública. Este projeto tem como objetivo desenvolver um sistema automatizado de monitoramento e análise de uso do parque de impressão da Secretaria Municipal de Educação, visando redução de custos operacionais e a otimização da gestão de suprimentos. A metodologia adotada fundamentou-se nos pilares do *Design Thinking*, estruturando-se nas etapas de compreensão do problema, idealização e prototipagem. Tecnicamente, a solução foi construída sobre o sistema operacional Linux Mint 22.2, utilizando a ferramenta Zabbix 6.0 para a comunicação primária com os equipamentos via protocolo SNMP. Foram desenvolvidos scripts automatizados em Python 3.12 para a extração diária dos níveis de toner e contadores de páginas, integrando-os a um banco de dados relacional na nuvem na plataforma Supabase (PostgreSQL).

PALAVRAS-CHAVE: Monitoramento de Ativos; Automação; Gestão Pública; Python; Análise de Dados.

SUMÁRIO

1 INTRODUÇÃO.....	5
2 DESENVOLVIMENTO.....	6
2.1 Objetivos.....	6
2.1.1 Objetivo geral.....	6
2.1.2 OBJETIVOS ESPECÍFICOS.....	6
2.2 Justificativa e delimitação do problema.....	6
2.3 Fundamentação teórica.....	7
2.4 Aplicação das disciplinas estudadas no projeto integrador.....	8
2.5 Metodologia.....	8
3 RESULTADOS: SOLUÇÃO FINAL.....	10
4 CONSIDERAÇÕES FINAIS.....	11
REFERÊNCIAS.....	12

1 INTRODUÇÃO

A redução do gasto e o melhor gerenciamento dos recursos sempre estão em pauta quando se trata da área pública, o bom uso dos recursos, sua melhor administração e além de entender melhor as necessidades são ações necessárias a serem analisadas antes de qualquer contrato ou compra.

O presente projeto propõe a criação de um projeto para análise do uso do parque de impressão visando entender melhor o uso por parte das Unidades Administrativas da Secretaria Municipal de Educação da Prefeitura Municipal de Jacareí.

Utilizando um servidor Linux equipado com *scripts* Python, Protocolo SNMP; PostgreSQL como banco de dados na nuvem e página Web para os *Dashboards*.

Dessa forma, o projeto busca contribuir para o melhor entendimento de uso e servir como uma base para fundamentar as reais necessidades das unidades administrativas, permitindo que futuras contratações utilizem estas informações.

2 DESENVOLVIMENTO

2.1 OBJETIVOS

2.1.1 OBJETIVO GERAL

O objetivo deste projeto consiste em desenvolver uma solução para captação dos dados gerados com o uso das impressoras lotadas nas Unidades Administrativas da Secretaria de Educação da Prefeitura Municipal de Jacareí, com o foco em processar estes dados a fim de entender melhor seu uso e os gastos gerados pelos equipamentos

2.1.2 OBJETIVOS ESPECÍFICOS

Levantar os gargalos e limitações do atual processo de gestão e uso de suprimentos de impressão na Secretaria de Educação de Jacareí.

Descrever a arquitetura de *software* implementada, detalhando a intergração entre os *scripts* de coleta, banco de dados e página Web.

Determinar regras de segurança para o acesso aos dados em nuvem, configurando políticas de nível de linha (*Row Level Security – RLS*) no banco de dados PostgreSQL.

Analisar o fluxo do consumo de suprimentos e a volumetria de páginas impressas por meio de um *dashboard*.

2.2 JUSTIFICATIVA E DELIMITAÇÃO DO PROBLEMA

Como norte para o desenvolvimento desta pesquisa e construção da solução, o grupo levou em mente a seguinte questão: Como a automação da coleta e análise de dados de impressora pode otimizar a gestão de suprimentos e embasar a tomada de decisão para redução de custos na Secretaria de Educação da Prefeitura Municipal de Jacareí?

Atualmente, o parque de impressões, que estão distribuídos entre as Unidades Administrativas e Unidades Escolares geram um volume significativo de dados (por exemplo, níveis de toner e contadores de páginas), que não são monitoradas ativamente, apenas coletadas e enviadas a

empresa contratada da locação dos equipamentos, resultando na falta de controle quanto ao uso e a perda de oportunidade em se entender melhor as reais necessidades das unidades.

Neste contexto, a justificativa do projeto se dá pela sua relevância administrativa, pois a implementação de um sistema para monitoramento garante maior transparência, previsibilidade e melhor alocação dos recursos públicos. Do ponto de vista acadêmico e tecnológico, o projeto demonstra a aplicação prática de conceitos de redes, banco de dados em nuvem e desenvolvimento Web.

2.3 FUNDAMENTAÇÃO TEÓRICA

O monitoramento de ativos de Tecnologia da Informação é um pilar essencial para a governança e redução de custos em ambientes corporativos e públicos. Ferramentas consolidadas como o *Zabbix* utilizam protocolos de rede padrão, como o SNMP (*Simple Network Management Protocol*), para realizar o controle e gerenciamento dos equipamentos (DIAS, ALVES, 2001). A documentação oficial do *Zabbix* (2022) destaca sua capacidade de nível empresarial para extrair dados vitais de *hardwares* em tempo real.

Durante a fase de configuração do monitoramento no *Zabbix*, buscou-se otimizar o tempo de implementação utilizando modelos comunitários, a exemplo do *Template Kyocera FS-KM-P Series* (ZABBIX, 2022). Contudo, durante os testes práticos, constatou-se que as métricas nativas deste modelo não contemplava a leitura dos níveis de toner. Para contornar essa limitação, foi empregada a ferramenta de diagnostico de rede *snmpwalk* para inspecionar a árvore de diretórios do equipamento. Através da varredura, foi possível mapear os *Object Identifiers* (OID) referentes ao valor máximo do toner (1.3.6.1.2.1.43.11.1.1.8.1.1) e ao valor restante do toner (1.3.6.1.2.1.43.11.1.1.9.1.1).

$$\left(\frac{\text{Valor restante de toner}}{\text{Valor máximo de toner}} \right) \times 100 = \text{Toner restante}\%$$

Em paralelo à coleta, a automação e o armazenamento dessas informações exigem tecnologias robustas. A utilização de rotinas automatizadas via agendadores de tarefas no sistema Linux, como o *crontab* (FARIAS, 2006), garante que a extração ocorra de forma agendada e imune de esquecimentos quando comparadas a coletas manuais. Para essas rotinas, a linguagem Python 3.12 foi escolhida devido a recentes otimizações de desempenho (PYTHON

SOFTWARE FOUNDATION, 2023), conectando-se de forma eficiente ao repositório de dados.

A aplicação de *Business Intelligence* (BI) e a construção de *dashboards* deixaram de serem ferramentas exclusivas da iniciativa privada e passaram a ser essenciais no setor público. A utilização de painéis de indicadores na gestão pública brasileira atua como ferramenta estratégica indispensável, pois consolida volumes complexos de informações em painéis visuais que facilitam a transparência, o monitoramento e, principalmente, a tomada de decisão rápida e baseada em evidências (FERREIRA, PEREIRA 2025).

O uso de banco de dados relacionais PostgreSQL, fornecidos pela plataforma Supabase, garante o armazenamento seguro do histórico como também a aplicação de regras rígidas de acesso aos dados (SUPABASE, 2026).

Relacionado a interface gráfica, a utilização do React.js em conjunto com o *framework* Next.js permitiu a construção de uma interface mais responsiva e que apresentasse com mais facilidade os dados. De acordo com Lazuardy e Anggraini (2022), o uso dessas tecnologias modernas permite que o desenvolvimento *frontend* se concentre exclusivamente na interface do utilizador, separando-o do *backend*.

2.4 APLICAÇÕES DAS DISCIPLINAS ESTUDADAS NO PROJETO INTEGRADOR

Infraestrutura de Sistemas de Software: Redes e Nuvem: Esta disciplina forneceu o embasamento necessário para a comunicação entre o sistema de monitoramento e o *hardware* das impressoras. Os conceitos de protocolos de rede foram aplicados na utilização do protocolo SNMP para a coleta de dados. Além disso, a configuração do ambiente em Linux Mint e a integração com o Subapase refletem os conceitos de infraestrutura e computação em nuvem estudada.

Visualização Computacional: Os princípios de abstração e representação visual de dados foram fundamentais para a criação do *Dashboard*. A escolha de gráficos de área e linhas, os usos de cores para indicar níveis críticos de suprimentos e a construção de uma interface que permite o *storytelling* dos dados são aplicações diretas desta disciplina, visando facilitar a tomada de decisão.

Desenvolvimento Web: Conhecimentos desta matéria foram aplicados na construção da interface utilizando o *framework* Next.js e a biblioteca React. A disciplina permitiu a estruturação da aplicação em componentes, o consumo de APIs para busca de dados em tempo real e a garantia de uma experiência de usuário fluida e eficiente.

Introdução a Ciências de Dados: Esta disciplina foi a base para o processamento e a inteligência do sistema. Através da linguagem Python, aplicaram-se técnicas de pré-processamento e limpeza de dados, essenciais para filtrar anomalias nos contadores das impressoras. Além disso, a lógica matemática para o cálculo da média mensal e da projeção anual de consumo baseia-se nos fundamentos de análise exploratória estudados.

2.5 METODOLOGIA

A pesquisa teve início a partir da observação do cenário do setor de Unidade de Tecnologias Educacionais da Secretaria de Educação de Jacareí. Através de levantamentos informais com a equipe de suporte do parque tecnológico, identificou-se a dificuldade de gerenciar ativamente as impressoras espalhadas pelas unidades administrativas e unidades escolares. A coleta de dados sobre a volumetria de impressão e os níveis de suprimentos era inexistente, o que deixava a unidade sem entender suas reais necessidades.

Com base nas informações levantadas, optou-se por uma arquitetura de *software* dividida em três camadas (*Backend*, Banco de Dados e *Frontend*). Para coleta de dados, foram desenvolvidos *scripts* automatizados na linguagem Python, integrados à ferramenta de monitoramento Zabbix. Como repositório central, utilizou-se o banco de dados relacional PostgreSQL, hospedado na plataforma em nuvem Supabase, organizando as informações em tabelas interligadas. Por fim, para a camada de visualização, foi projetado um *dashboard* web utilizando o *framework* Next.js (React), focado em fornecer uma interface limpa e intuitiva para os gestores.

A fase de implementação constituiu em colocar os *scripts* de coleta em produção em um servidor Linux, utilizando agendamento automatizado (*crontab*) para rodar diariamente. Durante os testes práticos com equipamentos reais, foram identificados e corrigidos cenários de falha, como impressoras offline que retornavam valores nulos ou zerado, e máquinas sem

identificação de nome no *Zabbix*. O código foi aprimorado com lógicas de validação (exigindo contadores estritamente maiores que zero) e métodos seguros de buscas de chaves.

Para a construção da solução, o ambiente base utilizado constituiu em um servidor rodando o sistema operacional Linux Mint 22.2. A escolha por esta distribuição específica e não uma versão focada em servidor como Ubuntu Server se deu pela presença de uma interface gráfica amigável, o que reduz a curva de aprendizado da equipe e facilitou a configuração de ferramentas de acesso remoto, permitindo o desenvolvimento assíncrono e fora das dependências da organização.

A infraestrutura tecnológica do projeto baseia-se nas seguintes versões de *software*: a coleta de dados primária é realizada pela ferramenta de monitoramento Zabbix 6.0, previamente instalada no ambiente. Os roteiros de automação (*scripts*) foram desenvolvidos na linguagem Python 3.12, garantindo compatibilidade com bibliotecas modernas de requisição e conexão. O banco de dados foi estruturado em PostgreSQL através da plataforma em nuvem Supabase. O *frontend* ficou por conta da utilização do Next.js (React) na plataforma Vercel, o que permite a construção dos *dashboards* com maior facilidade.

3 RESULTADOS: SOLUÇÃO FINAL

Para a solução final, foi garantida a criação do *script* para fazer a coleta diária dos dados de contadores e porcentagem dos toners, levando em consideração que dentro das unidades administrativas parte das informações já são conhecidas, como a quantidade de impressoras e IPs de rede das impressoras, o que já permite um monitoramento básico, além de haver um pequeno servidor Zabbix utilizado para a coleta dos dados.

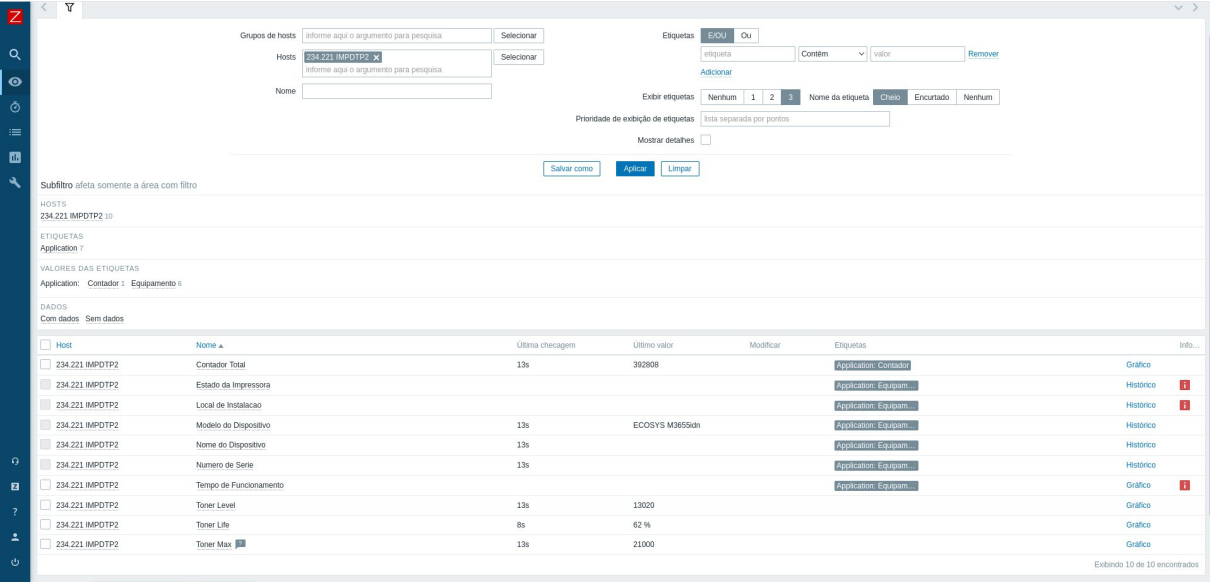
Além disso, esta etapa e todos os códigos referentes ao *frontend*, criou-se um repositório na plataforma *GitHub*. O objetivo deste repositório é centralizar os códigos desenvolvidos, facilitando a colaboração, versionamento das atualizações e servindo de consultas quanto a veracidade dos conteúdos apresentados neste projeto.

Utilizando o modelo comunitário *Template Kyocera FS-KM-P Series* (ZABBIX, 2022), notou-se que ainda não existia uma contagem em porcentagem do valor restante dos toners, neste caso, através da ferramenta *snmpwalk*, foi possível mapear os seguintes *Object Identifiers* (OIDs) de valor máximo do toner (1.3.6.1.2.1.43.11.1.1.8.1.1) e o valor restante do toner (1.3.6.1.2.1.43.11.1.1.9.1.). Para entender melhor a descrição, ocorre o seguinte: As máquinas da Kyocera não guardam informações sobre valores restante dos toners em porcentagem, toda vez que se instala um toner na máquina, ela pega o valor máximo de páginas que espera-se que seja impressas do toner, por exemplo, 20.000 páginas e conforme utiliza-se a máquina, ela retira o número de páginas impressoras, por exemplo, 6.000. Gerando assim o valor restante do toner $20.000 - 6.000 = 14.000$. Como o sistema só traz valores inteiros, o cálculo fica por conta do sistema, conforme o exemplo abaixo:

$$\left(\frac{14000}{20000}\right) \times 100 = 70\%$$

A imagem abaixo demonstra um equipamento sendo monitorado pelo Zabbix, contendo por exemplo, os valores que estão sendo retornados pelos OIDs cadastrados.

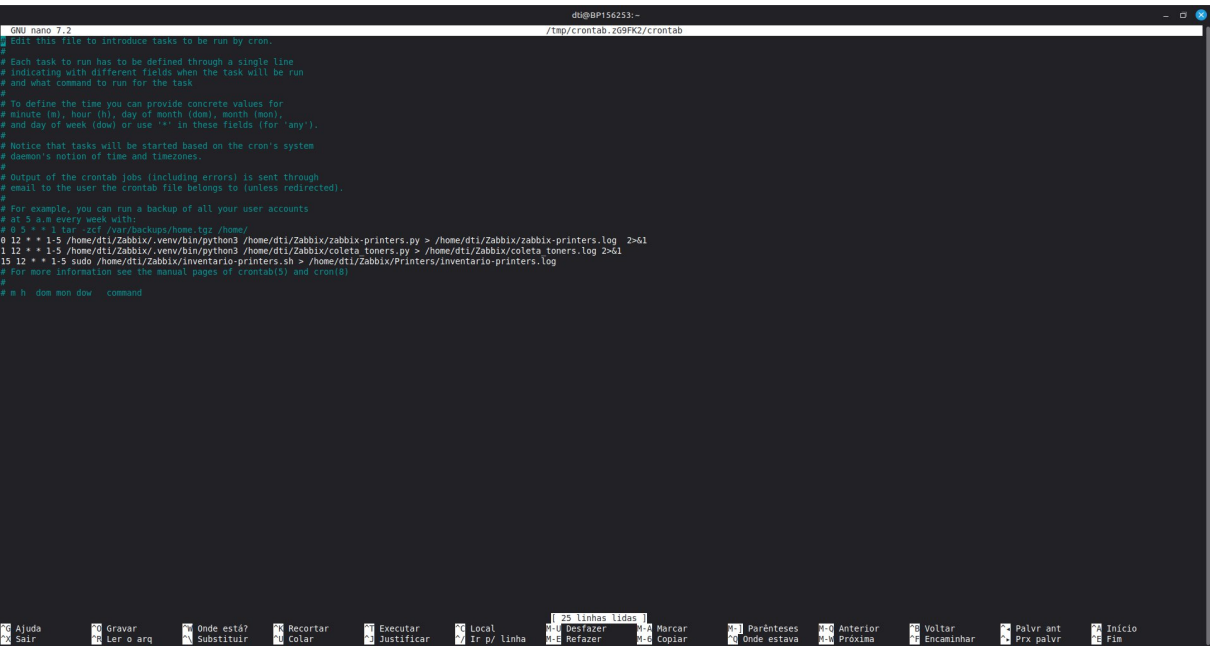
Figura 1 – Exemplo do painel do Zabbix.



Fonte: Autores

Para rodar o script em Python, fora criado uma tarefa utilizando o *crontab*, utilizando o comando “crontab –e” no terminal, a figura 2 traz a tela apresentada.

Figura 2 – Tela crontab do servidor Linux.



Fonte: Autores

Para entender melhor, abaixo está a mesma imagem, porém cortada para apresentar melhor o comando:

Figura 3 – Zoom aplicado na imagem crontab do Linux.

```
GNU nano 7.2 /tmp/crontab.zG9FK2/cr
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
0 12 * * 1-5 /home/dti/Zabbix/.venv/bin/python3 /home/dti/Zabbix/zabbix-printers.py > /home/dti/Zabbix/zabbix-printers.log 2>&1
1 12 * * 1-5 /home/dti/Zabbix/.venv/bin/python3 /home/dti/Zabbix/coleta_toners.py > /home/dti/Zabbix/coleta_toners.log 2>&1
15 12 * * 1-5 sudo /home/dti/Zabbix/inventario-printers.sh > /home/dti/Zabbix/Printers/inventario-printers.log
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
```

Fonte: Autores

O comando *crontab* é usado para criar e monitorar *jobs* (FARIAS, 2006). Para entender o comando, vamos pegar de exemplo a primeira linha em branco:

Figura 4 – Exemplo de linha do crontab do Linux.

```
0 12 * * 1-5 /home/dti/Zabbix/.venv/bin/python3 /home/dti/Zabbix/zabbix-printers.py > /home/dti/Zabbix/zabbix-printers.log 2>&1
```

Fonte: Autores

A seguir, a figura 5 traz uma explicação de cada item presente na linha do crontab.

Figura 5 – Explicação sobre o *crontab*.

- **0** – Minuto
- **12** – Hora
- ***** – Dia do mês
- ***** – Mês
- **1-5** – Dia da semana.
- **/home/dti/Zabbix/.venv/bin/python3** – Local onde está as bibliotecas Python
- **/home/dti/Zabbix/zabbix-printers.py** – Script Python para verificar as impressoras e retirar os contadores
- **>**– Sinal utilizado para pedir ao Linux que toda vez que rodar o programa, tudo que aparecer no terminal seja escrito no arquivo apresentado na linha seguinte.
- **/home/dti/Zabbix/zabbix-printers.log** – Arquivo gerado com os logs gerados pelo script.

Fonte: Autores

Explicando o conteúdo: De segunda a sexta, as 12h, o Python irá entrar em `/home/dti/Zabbix/.venv/bin/python3` e executar `zabbix-printers.py`, durante a execução, os valores gerados no console irão ser armazenados em `zabbix-printers.log` e se já existirem dados escritos, serão substituídos, mantendo somente os *logs* atuais do código.

As estruturas dos arquivos ficaram da seguinte maneira:

Figura 6 – Estrutura usada pelo projeto no servidor

```
/home/dti/Zabbix/  
— .env # Arquivo com as credenciais (Supabase e Zabbix)  
— coleta_toners.py #Script para coleta de níveis de suprimentos  
— zabbix-printers.py #Script para coleta dos contadores de páginas  
— coleta_toners.log #Arquivo de log da última execução de coleta_toners.py  
— zabbix-printers.log #Arquivo de log da última execução de zabbix-printers.py
```

Fonte: Autores

O arquivo `zabbix-printers.py` procura o sistema do Zabbix por qualquer máquina que esteja no grupo IMPRESSORAS e procurar pelas informações de contadores. Conforme imagem abaixo:

Figura 7 – Código parcial do arquivo `zabbix-printers.py`

```
ZABBIX_URL = os.getenv("ZABBIX_URL", "http://127.0.0.1/zabbix/api_jsonrpc.php")  
ZABBIX_TOKEN = os.getenv("ZABBIX_TOKEN")  
  
NOME_GRUPO_ZABBIX = "IMPRESSORA"  
  
NOMES_DOS_CONTADORES = [  
    "Contador Total",          # Kyocera M3655idn  
    "Pages printed - total",   # Xerox C505s  
    "Page Counter",           # Brother L6902W e L6912W  
    "TotalPrits"               # Konica 558e e 645e  
]
```

Fonte: Autores

O conteúdo da variável `NOME_DOS_CONTADORES` contém as chaves usadas dentro do Zabbix pelos *templates* das impressoras, cada template contém seu próprio nome, conforme comentado no código. **Contador Total** para as impressoras Kyocera pegas pelo *Template Kyocera FS-KM-P Series*, **Pages printed - total** para a impressora Xerox e retirada do

Template Printer Xerox WorkCentre 3220, **Page Counter** para as impressoras Brother e retirado do template *Brother Printers* e **TotalPrits** para as impressoras Konica e retirado do *template Printer_Konica*.

Utilizando os valores acima e já tendo pegado os IDs dentro do Zabbix para as impressoras, o sistema então passa a fazer a coleta dos valores salvos até ao meio dia para cada item, conforme o código abaixo:

Figura 8 – Código da coleta dos contadores.

```
# =====
# COLETA DOS CONTADORES
# =====
print("\n-> Iniciando a coleta de contadores...")
for z_id, imp in mapa_impressoras.items():

    # Cria variáveis blindadas contra o KeyError
    nome_seguro = imp.get('nome_maquina') or "Sem Nome"
    ip_seguro = imp.get('endereço_ip') or "IP Desconhecido"
    modelo_seguro = imp.get('modelo_impressora') or "Modelo Desconhecido"

    valor_contador, nome_item = buscar_contador_zabbix(z_id)

    # Garante que o contador não é nulo E é maior que 0
    if valor_contador is not None and valor_contador > 0:
        dados_contador = {
            "id_impressora": imp['id'],
            "qtd_contador": valor_contador
        }

        try:
            supabase.table("tabelaContador").insert(dados_contador).execute()
            print(f"✅ SUCESSO: '{nome_seguro}' (IP: {ip_seguro}) [{nome_item}] - Contador: {valor_contador}")
        except Exception as e:
            print(f"❌ Erro ao salvar contador da '{nome_seguro}' (IP: {ip_seguro}): {e}")

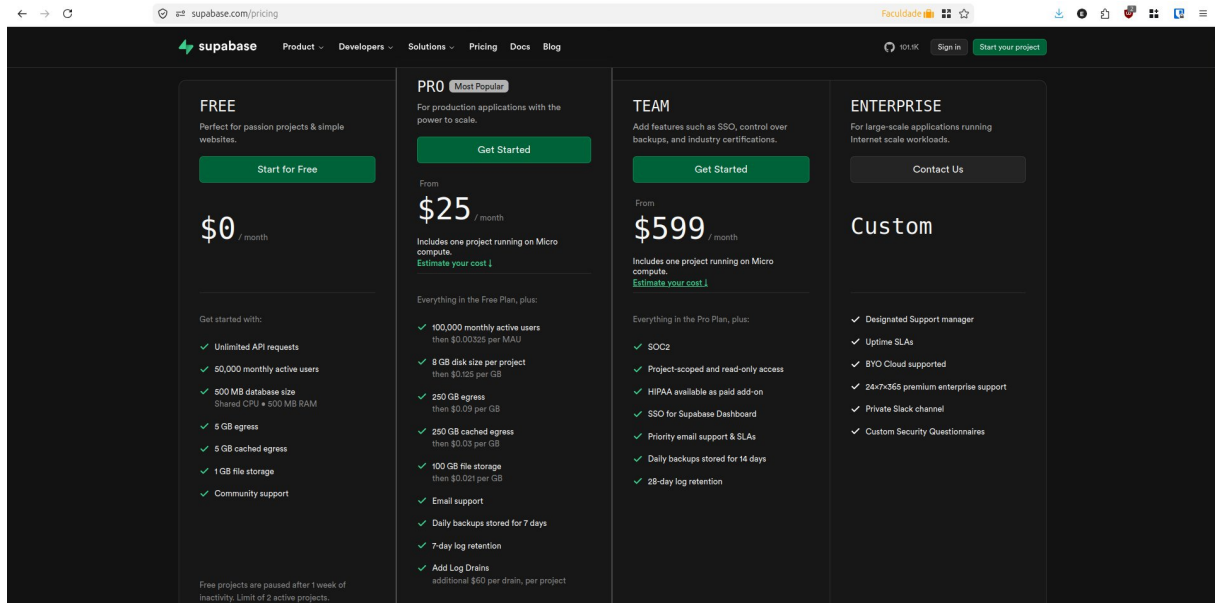
    else:
        # Pula suavemente sem dar erro de KeyError
        print(f"⚠️ AVISO: Impressora '{nome_seguro}' (IP: {ip_seguro}) offline ou contador zerado. Pulando...")
        continue
```

Fonte: Autores

Vale notar que caso o valor venha zerado ou nulo, o código irá pular, criando um resultado descrito o nome da impressora e o IP, gerando um texto parecido com: “AVISO: Impressora Xerox IP: 220 offline ou contador zerado. Pulando...”

Já pelo banco de dados em nuvem, o Supabase em sua versão gratuita, até a criação deste projeto, permite aos usuários até 500MB de armazenamento na nuvem para as tabelas do banco de dados, visto que o projeto irá armazenar somente os dados e depois irá fazer a análise pelo Vercel, o limite consegue suprir as demandas atuais. Conforme imagem abaixo, vemos as limitações presentes:

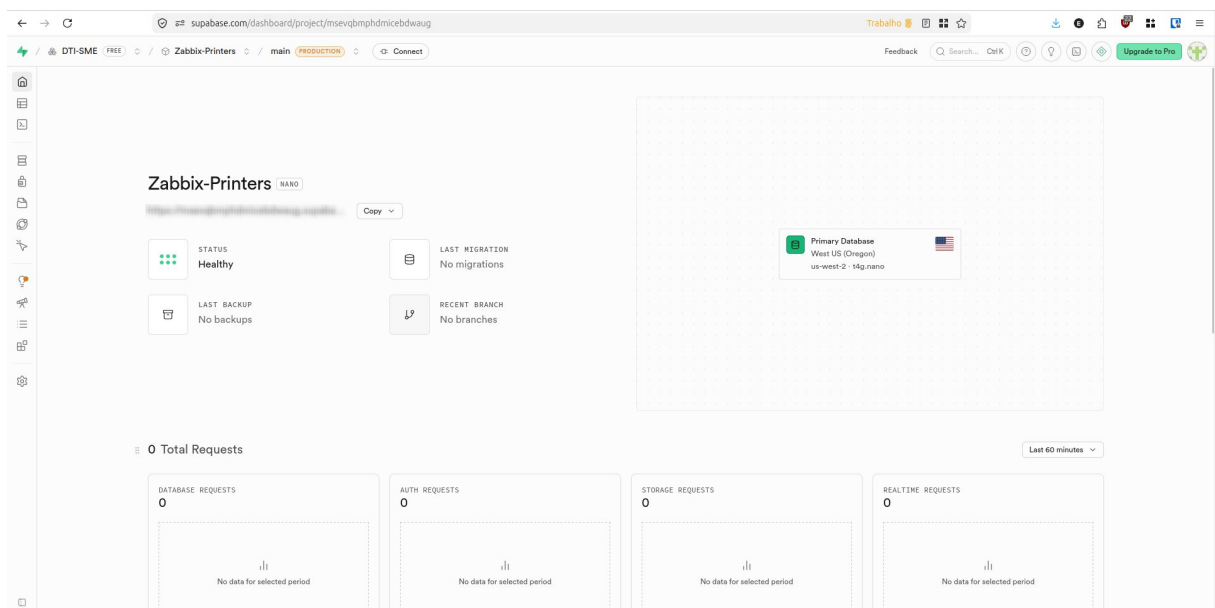
Figura 9 – Imagem listando os pacotes disponíveis do Supabase.



Fonte: Supabase

Na tela inicial dos projetos, figura 10, o Supabase traz informações básicas como o nome do projeto, o link de acesso direto e dashboards quanto as requisições e acessos no dia em questão.

Figura 10 – Captura de tela do painel principal do projeto.



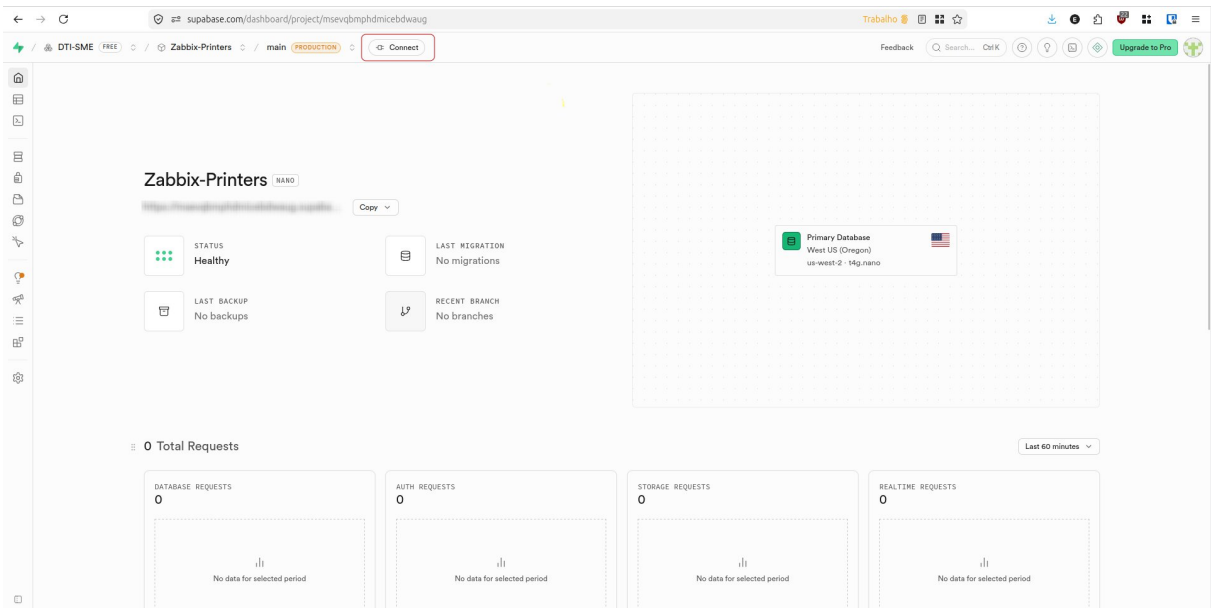
Fonte: Supabase

O nome do projeto utilizado é Zabbix-Printers, o modo de funcionamento do Supabase permite que o acesso ao banco de dados seja feito diretamente por JavaScript, sendo

necessário somente ter acesso aos *links* de autenticação, para acessar os componentes, ele é feito da seguinte maneira.

1º Clicar em **Connect** no menu superior do site, conforme imagem abaixo:

Figura 11 – Captura de tela do painel do projeto com instruções



Fonte: Supabase

2º Uma página lateral irá abrir, nela é possível escolher a forma como será feita a conexão. No caso deste projeto, ele será em Next.js.

Figura 12 – Opções de conexão com o banco de dados.

Connect to your project

Choose how you want to use Supabase

Framework

Use a client library

Direct

Connection string

ORM

Third-party library

MCP

Connect your agent

Framework

Variant

Shadcn

Next.js

App Router

Install components via the Supabase shadcn registry.

Connect your app

Give your agent everything it needs

Copy prompt

1

Install packages

Run this command to install the required dependencies.

npm install @supabase/supabase-js @supabase/ssr

2

Add files

Add env variables, create Supabase client helpers, and set up middleware to keep sessions refreshed.

.env.local

page.tsx

utils/supabase/server.ts

utils/supabase/client.ts

utils/supabase/middleware.ts

NEXT_PUBLIC_SUPABASE_URL=

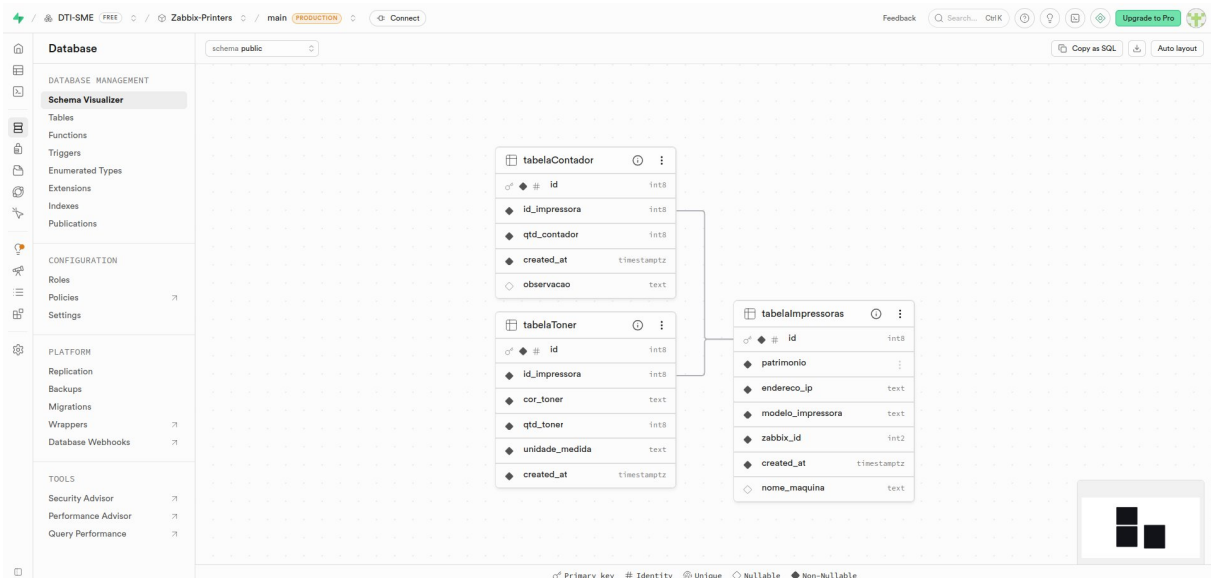
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=

Fonte: Supabase

Utilizando o arquivo salvo no projeto em Next.js é possível estar fazendo a conexão com o banco de dados, por se tratar de um arquivo oculto e salvo apenas no *framework*, as informações ficam públicas apenas aos *dashboards* do projeto, evitando acesso direto por terceiros.

Em relação ao banco de dados, o projeto utiliza apenas três tabelas, conforme demonstrado na imagem abaixo. Isso garante que o projeto mantenha uma lógica simples, facilitando a manutenção futura conforme necessário.

Figura 13 – Modelagem do banco de dados



Fonte: Autores

Sendo composto por tabelaContador, tabelaImpressora e tabelaToner, cada tabela cotendo seu ID como identificado primário e as tabelaContafor e tabelaToner contendo a identificação da impressora como chave secundária, criando suas relações e auxiliando a pesquisa dos dados.

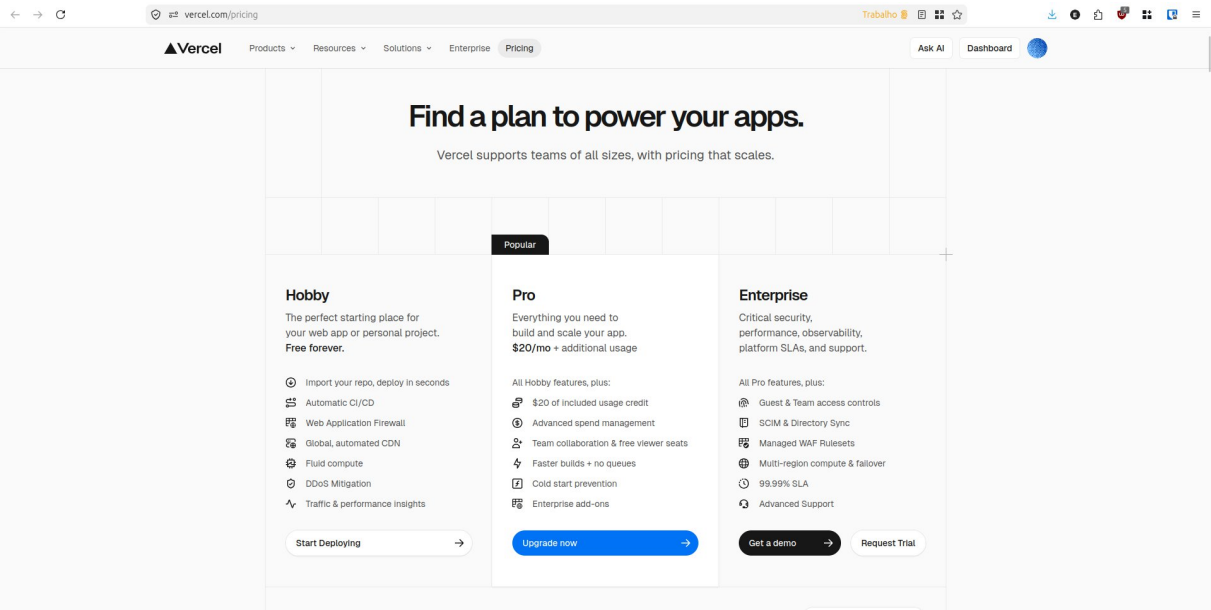
Tabela de Contadores: Toda vez que um contador é cadastrado, o banco de dados irá criar uma identificação (id), receber o a identificação (id_impresora) da impressora, pegar os valores recebidos e preencher em qtd_contador. Por segurança e facilidade de identificação, o campo created_at é preenchido automaticamente pelo banco de dados com a hora atual da inscrição do contador.

Tabela de Toner: Toda vez que um toner é cadastrado, o banco gera a identificação única (id), recebe a identificação da impressa (id_impresora), pega a cor em questão do toner (cor_toner), quantidade naquele momento (qtd_toner) e a Unidade a ser utilizada (unidade_medida), visto que nem sempre os valores podem estar vindo em porcentagem. Quanto ao campo created_at, assim como o caso anterior, ele é preenchido automaticamente pelo banco de dados.

Tabela de Impressoras: Cadastro geral das impressoras no sistema, sua identificação (id) é a principal do projeto, é através dele que todo o resto irá funcionar, o campo patrimonio se refere ao número de patrimônio que ela recebe pela empresa terceirizada, a principal distinção entre elas será o campo modelo_impressora, onde será salvo o nome do modelo (por exemplo, M3655idn). O campo created_at assim como anteriormente é preenchido anteriormente pelo banco de dados.

Para a análise de dados e *dashboards*, utilizou-se então a plataforma Vercel e sua opção para contas *Hobby*, focadas em pequenos projetos. Esta versão permite então a criação da aplicação Web utilizando Next.js.

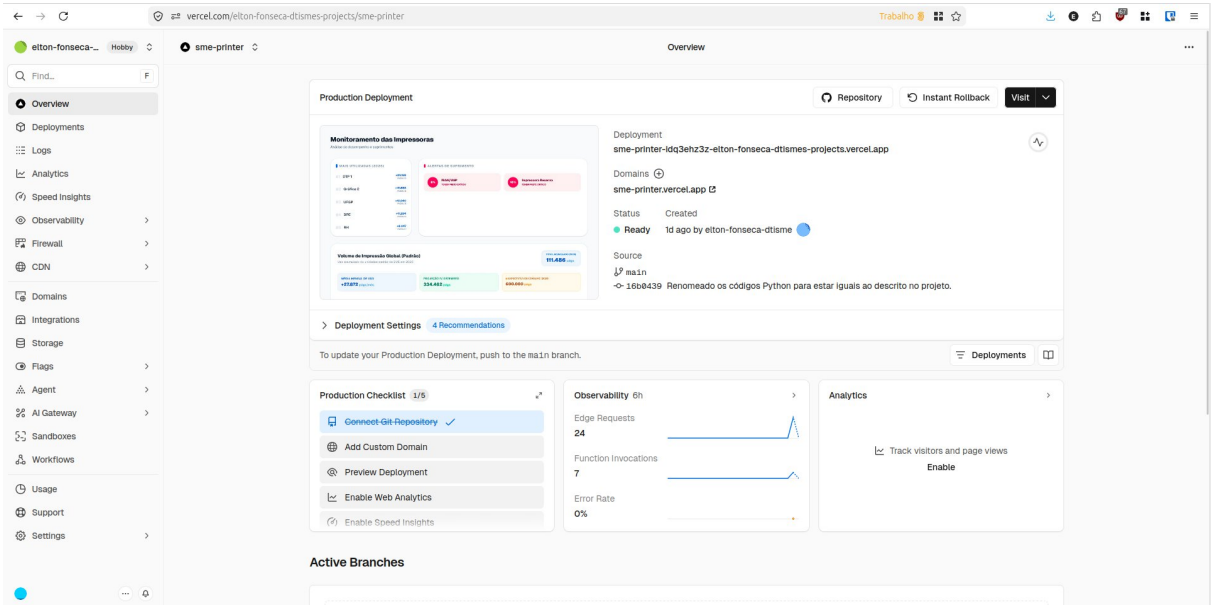
Figura 14 – Captura de tela com os planos disponíveis do Vercel.



Fonte: Vercel

Acessando as telas do Vercel, o projeto sme-printer é apresentado conforme a imagem abaixo.

Figura 15 – Tela do projeto no Vercel

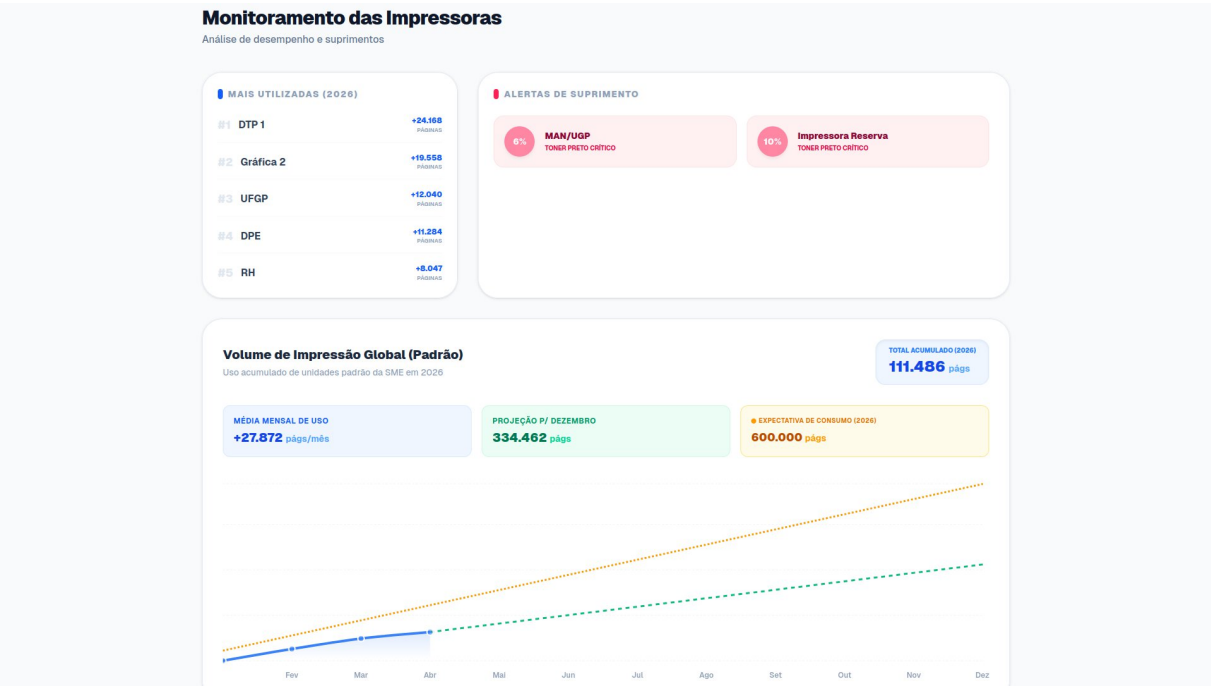


Fonte: Vercel

Como é possível observar, a página inicial traz informações referentes ao *deploy* do projeto, como o link do domínio, a fonte do projeto, neste caso, ele é gerado toda vez que o *branch* *main* do GitHub é atualizado.

Para os *dashboards*, a seguir serão apresentados os exemplos práticos utilizados nas aplicações Web que estão disponíveis no seguinte link: <https://sme-printer.vercel.app/>. A tela inicial da aplicação ficou conforme apresentado na figura 16.

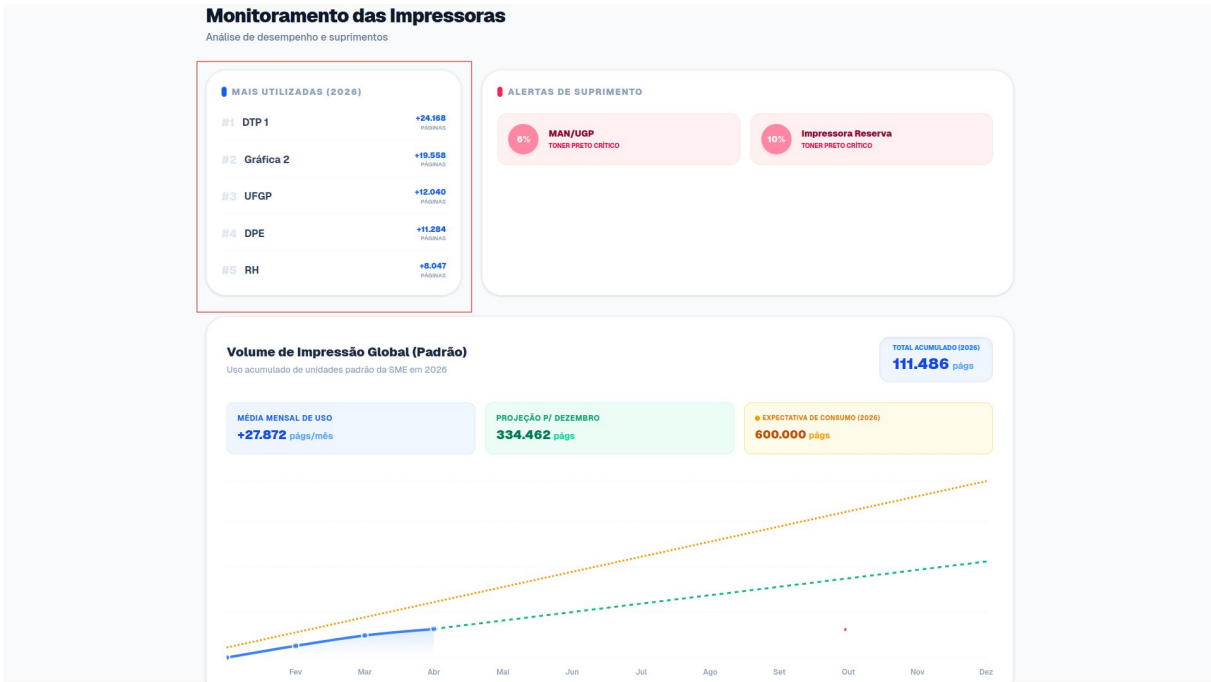
Figura 16 - Captura da tela inicial do *Dashboard* do projeto.



Fonte: Autores

A seguir, serão apresentadas mais informações relacionadas aos componentes disponíveis para visualização na aplicação Web, bem como uma explicação de suas funcionalidades e motivações.

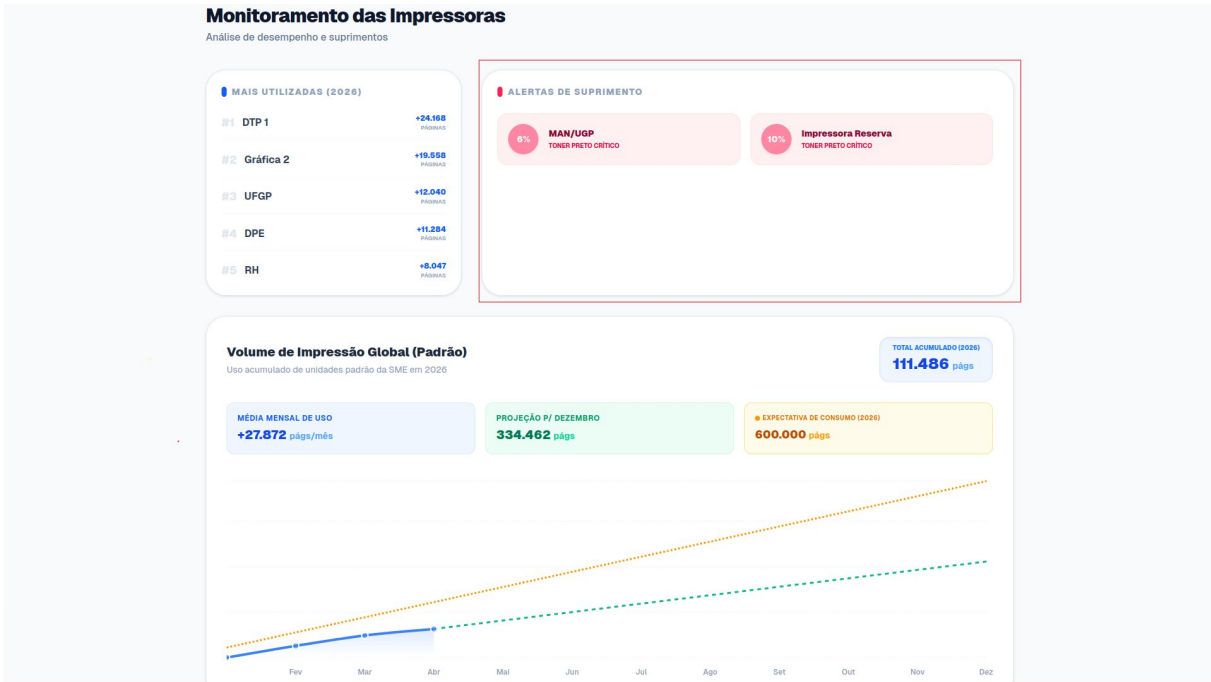
Figura 17 – Captura da tela inicial do *Dashboard*, marcando o campo “Mais Utilizado”.



Fonte: Autores

No canto superior esquerdo a aplicação Web traz uma lista das impressoras que foram mais utilizadas até o presente momento, trazendo sua opção em uma lista do 1º ao 5º colocado e sua quantidade de páginas impressoras, como no exemplo, a impressora chamada DTP 1 teve um uso de 24.168 até a data da imagem 21/04/2026.

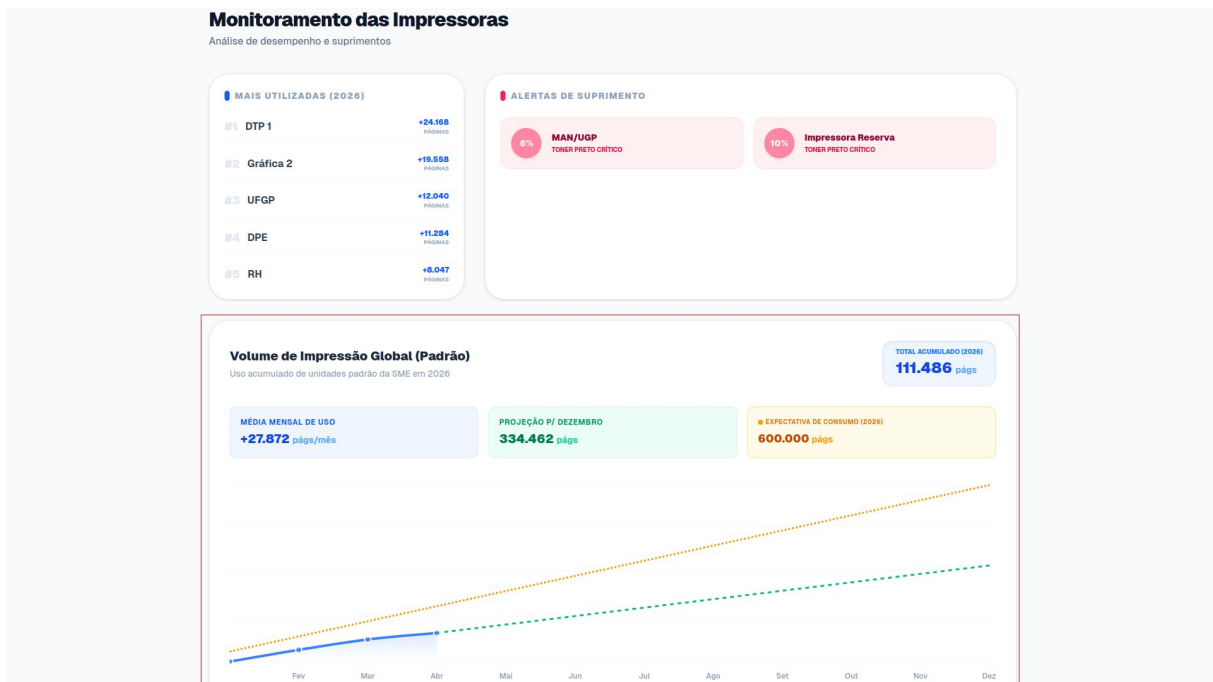
Figura 18 – Captura de tela do *Dashboard*, marcando a área “Alerta de Suprimentos”



Fonte: Autores

Ao lado, é possível encontrar o **ALERTA DE SUPRIMENTO**, a principal intenção neste caso é trazer as impressoras que estão com o toner baixo, deixando claro que estes dispositivos em algum momento irão requisitar novos suprimentos.

Figura 19 – Captura de tela do *Dashboard*, marcando a área “Valores de Impressão Global”.

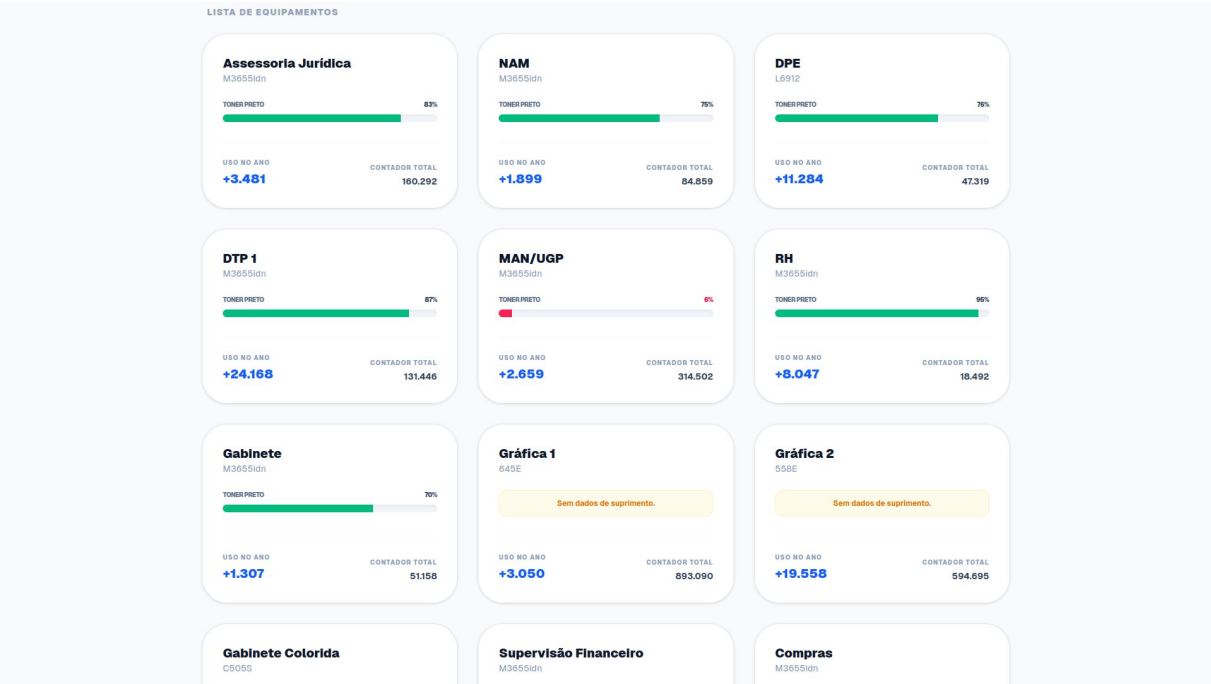


Fonte: Autores

As informações apresentadas em **Volume de Impressão Global**, trazem os valores acumulados até o momento de todas as impressoras que estão sendo monitoradas atualmente. Além disso, há neste caso a análise dos dados quanto ao consumo. No topo do *dashboard*, é possível ver as informações **Média Mensal de Uso**, fazendo a coleta dos últimos valores de cada mês e criando então sua média e apresentando no gráfico em azul. Ao lado, a informações em **PROJEÇÃO P/ DEZEMBRO** é feita utilizando o valor anterior da média por mês e projetando este mesmo acréscimo a cada mês, criando, por exemplo, a projeção para Dezembro para mais de 300000 páginas. Já o valor pego em **EXPECTATIVA DE CONSUMO** é retirado diretamente da planilha de contadores do ano anterior, fazendo isso, espera-se que o consumo real anual seja apresentado até o final do ano.

Abaixo destas informações, a página foi projetada para apresentar as informações mais atuais de cada impressora, conforme abaixo, no grupo **LISTA DE EQUIPAMENTOS**.

Figura 20 – Captura de tela do projeto, na área dos equipamentos individuais.

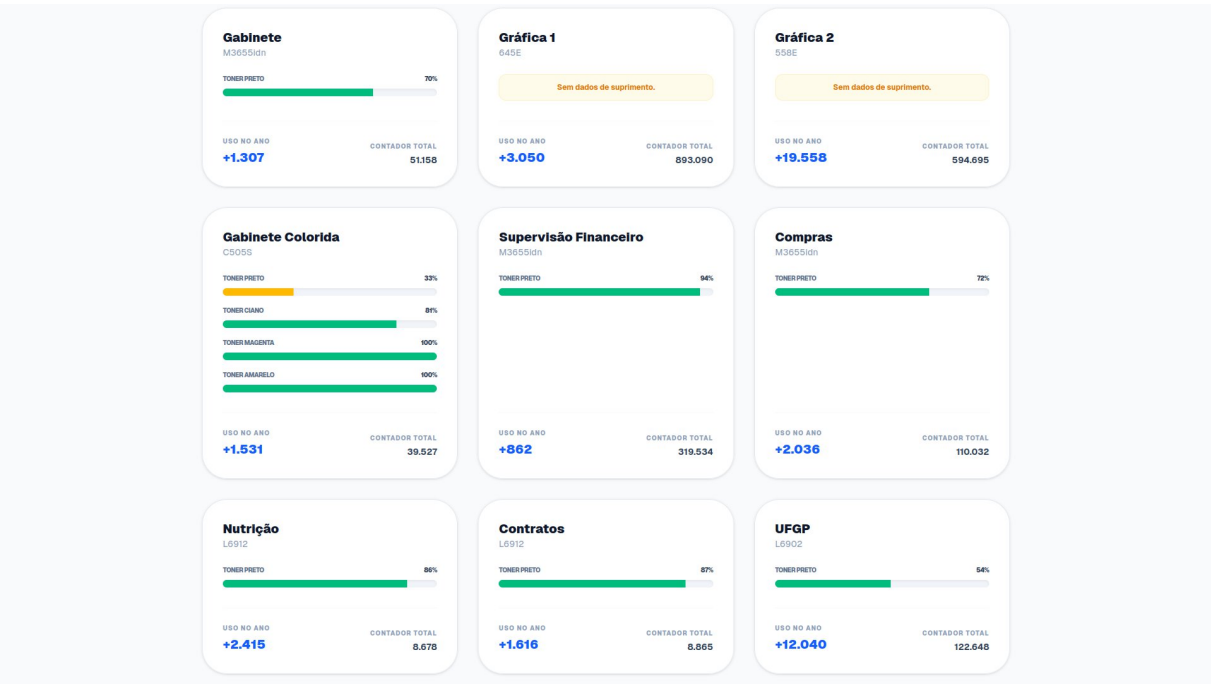


Fonte: Autores

Em cada cartão, é apresentado então as informações: Local da impressora, Modelo da impressora, Informações do Toner, Uso da equipamento no canto inferior esquerdo e o valor do contador ao lado direito.

Uma observação sobre o resultado fica em relação a impressora Colorida, para garantir a integridade da análise de consumo, aplicou-se uma regra no código que segrega equipamentos com contratos diferentes, garantindo que a média mensal e projeção global reflitam apenas ao maquinário comum e não sofram distorções pelo uso de valores que não pertencem ao grupo. Além disso, seu cartão apresenta as informações do toner Preto, Ciano, Magenta e Amarelo, ajudando na análise de uso e manutenção preventiva do equipamento.

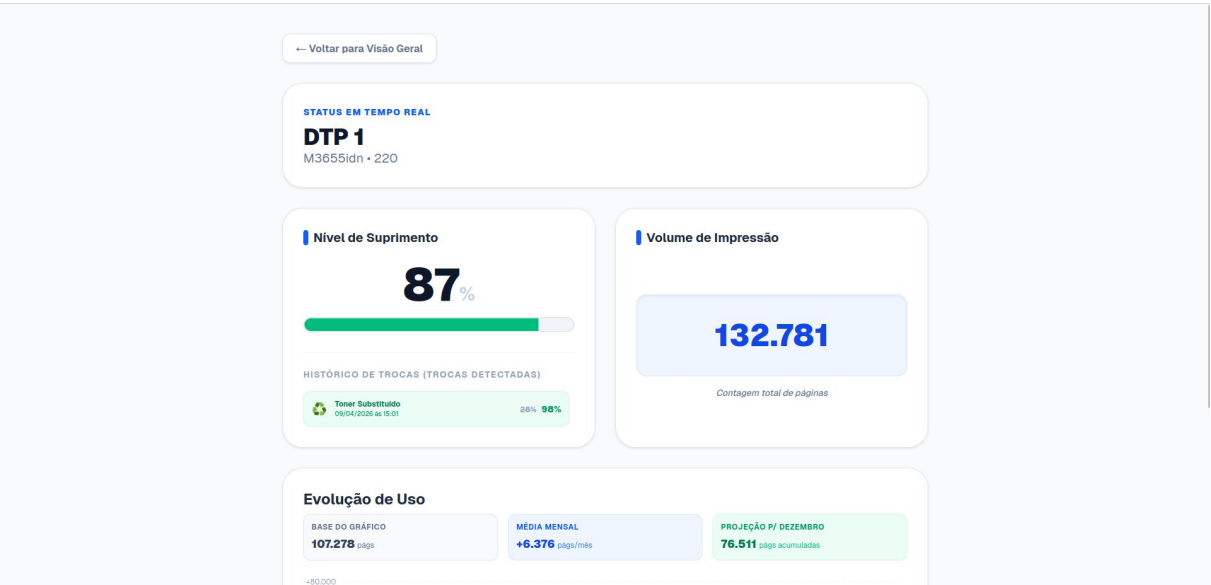
Figura 21 – Captura de tela do *Dashboard*, com foco no equipamento “Gabinete Colorida”.



Fonte: Autores

Durante o acesso a interface Web do projeto, é possível clicar em qualquer um dos *cards* apresentado e ter acesso a informações individualizadas, conforme a Figura 22, as informações apresentadas são: Nível de Suprimento, Volume de impressão, Modelo da impressora, Histórico de trocas de toner e Evolução de Uso.

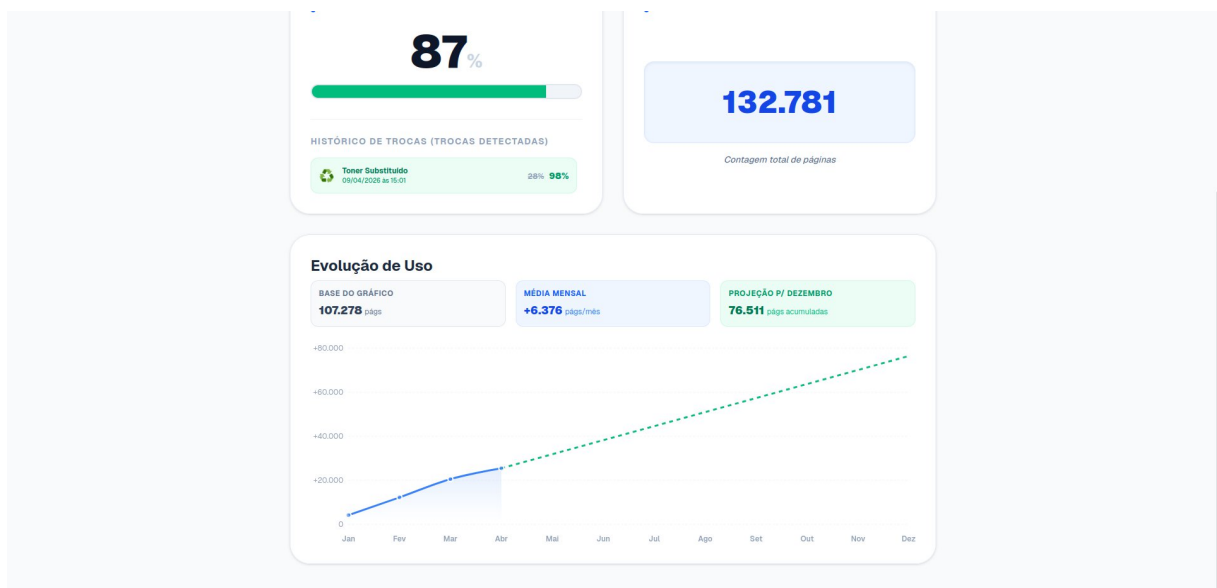
Figura 22 – Captura de tela *Dashboard* de equipamentos individuais.



Fonte: Autores

Nesta página, as informações apresentadas serão da impressora individual, permitindo um entendimento melhor quanto ao uso do equipamento, facilitando a consulta de contadores quando necessário, entender o histórico da última troca dos toners. Na Figura 23, é possível ver o exemplo do gráfico contido na parte inferior da página.

Figura 23 – Captura de tela *Dashboards* de equipamentos individuais com foco no gráfico inferior.



Fonte: Autores

Utilizando assim a mesma ideia da do gráfico global, a Evolução de Uso traz uma versão visual para apresentar como está o uso do equipamento a cada mês, possibilitando entender se o uso segue um padrão ou se houve um uso acima da média no último mês. Para cada mês, o valor utilizado será sempre o último valor coletado no mês e, caso esteja no mês atual, o valor coletado é o enviado no último dia ao banco de dados.

4 CONSIDERAÇÕES FINAIS

Este projeto integrador alcançou o seu objetivo geral de desenvolver um sistema automatizado para a captação e análise de dados do parque de impressão da Secretaria de Educação da Prefeitura Municipal de Jacareí. A transição de um modelo de coleta manual e reativo para uma arquitetura automatizada provou ser uma solução viável, segura e de baixo custo.

A principal contribuição deste trabalho é a entrega de transparência e previsibilidade à gestão pública. Com a implementação do *Dashboard*, os gestores agora possuem ferramentas visuais baseadas em evidências de consumo baseadas no ano anterior e antecipar a troca de suprimentos críticos antes que as unidades sejam impactadas. As limitações encontradas, como erros de leitura de contadores e a diferenciação de contratos, foram mitigadas com sucesso através de tratamentos de dados (*Data Cleansing*) e aplicação de regras de negócios no *frontend*.

Trabalhos futuros:

Como impacto na comunidade e próximos passos para evolução da ferramenta, segure-se a implementação de novas análises preditivas baseadas na base de dados agora estabelecidas, tais como:

1. **Monitoramento de Saúde (*Offline Tracking*):** Criação de alertas de ociosidade para identificar máquinas desconectadas da rede há vários dias.
2. **Análise de Rendimento de Toner:** Cruzar a volumetria de páginas impressoras com a queda percentual do toner para auditar se os suprimentos estão rendendo conforme estipulado em contrato, além de possibilitar a previsão de sua troca.

REFERÊNCIAS

DIAS, Beethovem Zanella; ALVES, Nilton. Protocolo de gerenciamento SNMP. Rio de Janeiro: Centro Brasileiro de Pesquisas Físicas, 2001. (Nota Técnica CBPF-NT-006/01).Disponível em: <https://cbpfindex.cbpf.br/publication_pdfs/nt00601.2010_10_15_11_40_12.pdf>. Acesso em: 5 abr. 2026.

FARIAS, Paulo Cesar Bento. Curso Essencial de Linux. São Paulo: Universo dos Livros,2006. Disponível em: <<https://books.google.com.br/books?id=IqRSiCofvUUC>>. Acesso em: 4abr. 2026.

FERREIRA, Mário Roberto; PEREIRA, Gilberto de Araújo. Utilização de painéis de indicadores (dashboards) na gestão pública brasileira: revisão integrativa. Observatório de laEconomía Latinoamericana, [S. l.], v. 23, n. 1, e8610, 2025. Disponível em: <<https://ojs.observatoriolatinoamericano.com/ojs/index.php/olel/article/view/8610/5439>>. Acesso em: 5 abr. 2026.

LAZUARDY, Mochammad Fariz Syah; ANGGRAINI, Dyah. Modern Front End Web Architectures with React.Js and Next.Js. International Research Journal of Advanced Engineering and Science, [S.I.], v.7,n.1,p. 132.141,2022. Disponível em: <<http://irjaes.com/wp-content/uploads/2022/02/IRJAES-V7N1P162Y22.pdf>>. Acesso em 26 abr. 2026.

NEXT.JS. Next.js Documentation. [S.I.], 2026. Disponível em: <<https://nextjs.org/docs>>.Acesso em: 26 abr. 2026.

SUPABASE. Supabase Documentation. [S. l.], 2026. Disponível em: <<https://supabase.com/docs>>. Acesso em: 4 abr. 2026.

PYTHON SOFTWARE FOUNDATION. Documentação Python 3.12.13. [S. l.], 2023.Disponível em: <<https://docs.python.org/pt-br/3.12/>>. Acesso em: 5 abr. 2026.

VERCEL.Vercel Documentation. [S.I.], 2026. Disponível em: <<https://vercel.com/docs>> Acesso em: 26 abr. 2026.

ZABBIX. Community Templates: Brother Printers. [S.l.]: GitHub, 2022. Disponível em: <https://github.com/zabbix/community-templates/tree/main/Printers/Brother/template_impressora_brother/6.0>. Acesso em: 4. Abr. 2026.

ZABBIX. Community Templates: Printer_Konica. [S. l.]: GitHub, 2022. Disponível em: <https://github.com/zabbix/community-templates/tree/main/Printers/Konica/template_konica_364_368_snmp/6.0>. Acesso em 4 abr. 2026.

ZABBIX. Community Templates: Template Kyocera FS-KM-P Series. [S. l.]: GitHub, 2022. Disponível em: <https://github.com/zabbix/community-templates/tree/main/Printers/Kyocera/template_kyocera_fs-km-p_series/6.0>. Acesso em: 5abr. 2026.

ZABBIX. Community Templates: Printer Xerox WorkCentre 3220. [S.l.]: GitHub, 2022. Disponível em: <https://github.com/zabbix/community-templates/tree/main/Printers/Xerox/template_xerox_workcentre_3220/6.0>. Acesso em: 4abr. 2026.

ZABBIX. Zabbix Documentation 6.0. [S. l.], 2022. Disponível em: <https://www.zabbix.com/documentation/6.0/downloads/Zabbix_Documentation_6.0.pt.pdf> Acesso em: 4 abr. 2026.